

Sber. Тестовое задание

Задание 1

N, M - произвольные числа, больше n . Поскольку для вычисления умножения матриц разрешается использовать только функцию f , напрашивается следующее решение: разделить исходную матрицу весов W на квадратные матрицы ранга n . Входящие N значения стоит делить таким же образом:

$$\begin{bmatrix} A_{11} & A_{12} & \dots & A_{1k} \\ A_{21} & A_{22} & \dots & A_{2k} \\ \vdots & \vdots & \ddots & \vdots \\ A_{l1} & A_{l2} & \dots & A_{lk} \end{bmatrix}$$

Где:

$$A_{ij} \in \mathbb{R}^{n \times n}$$

Исходная матрица делится на матрицы A_{ij} , если имеет соответствующие размерности: $N, M \bmod n = 0$. В случае несоблюдения этого условия, исходную матрицу весов можно дополнить нулями до момента, когда её размерность будет удовлетворять этому условию.

Каждый блок вычисляется с помощью функции f , которая будет принимать на вход часть весов и соответствующую ей часть входящих значений:

$$f(A_{ij}, x_j)$$

Следовательно:

$$y_i = \sum_{j=1}^k f(A_{ij}, x_j)$$

Где y_i - это блок из n значений.

Итоговый вектор y - конкатенация всех y_i .

Вычисление градиентов

Раскрывая f , y_i вычисляется следующим образом:

$$y_i = \sum_j W_{ij} x_j$$

Градиенты по весам

$$\frac{\partial y_i}{\partial W_{ij}} = x_j$$

В случае градиента y_i по W_{kl} , где $k \neq i$:

$$\frac{\partial y_i}{\partial W_{kl}} = 0$$

Поскольку такой элемент w_{kl} не влияет на значение y_i .

Градиент по входам

Вычисление градиента по входам довольно прямолинейное и, в отличие от градиента по весам, не содержит разнообразных случаев.

$$\frac{\partial y_i}{\partial x_j} = W_{ij}$$

Задание 2

Для уменьшения количества параметров было 2 идеи:

- факторизация линейного слоя: представления матрицы W в виде произведения двух матриц, каждая из которых имеет ранг r , такой, что $M * r + N * r < N * M$;
- дублирование весов для некоторых входов - например, зафиксировать, чтобы второй вход второго нейрона всегда имел такое же значение, как первый нейрон первого;

Второй подход достаточно прямолинейный (и скучный), поэтому было решено использовать первый подход.

В этом случае линейный слой представляется 2 матрицами:

$$U \in \mathbb{R}^{N \times r}, \quad V \in \mathbb{R}^{r \times M}$$

Тогда общее количество параметров будет достигать $N * r + M * r$. Можно найти r , при котором общее количество параметров уменьшится:

$$(N + M) * r < M * N$$

$$r < \frac{M * N}{M + N}$$

Выход линейного слоя вычисляется следующим образом:

$$y = UVx$$

Поскольку требуется использовать функцию f , принцип решения будет схож с принципом решения из Задания 1, но теперь такую операцию необходимо будет выполнить с 2 матрицами:

$$h_i = \sum_j f(V_{ij}, x_j)$$

$$y_i = \sum_j f(U_{ij}, h_j)$$

Вычисление градиентов

Градиенты по весам

По аналогии градиента из Задания 1:

$$\frac{\partial y_i}{\partial U_{ab}} = \begin{cases} h_b & \text{при } a = i \\ 0 & \text{при } a \neq i \end{cases}$$

При раскрытии изначальной формулы:

$$y_i = \sum_{k=1}^r \sum_{j=1}^N U_{ik} V_{kj} x_j$$

Тогда:

$$\frac{\partial y_i}{\partial V_{ab}} = \sum_{k=1}^r \sum_{j=1}^N U_{ik} x_j \frac{\partial V_{kj}}{\partial V_{ab}}$$

$$\frac{\partial y_i}{\partial V_{ab}} = U_{ia} x_b$$

Градиенты по входу

$$\frac{\partial y_i}{\partial x_b} = \sum_{k=1}^r U_{ik} V_{kb} = (UV)_{ib}$$

То есть:

$$\frac{\partial y}{\partial x} = UV$$

Как оценить качество работы такого слоя? В чём преимущества?

Представление линейного слоя в таком виде естественным образом ухудшает его возможную точность: теперь линейный слой не способен выразить любое линейное преобразование.

Тем не менее, у такого слоя есть несколько преимуществ: меньшее количество параметров занимает меньший объём памяти, благодаря небольшому рангу будет произведено меньше операций, что напрямую влияет на скорость работы. Это также может являться своего рода регуляризацией для линейного слоя: изменения одного параметра затрагивает сразу несколько входов разных нейронов, а значит модели потребуется обобщать свои знания.

К получившемуся линейному слою применимы все те же методы оценки, что и к обычному, метрики будут зависеть от задачи. К такой факторизации имеет смысл прибегать только в том случае, если требуется оптимизировать модель с точки зрения памяти и/или скорости (поскольку обычная L1/L2 регуляризация работает очень хорошо с линейным слоем без использования такого метода). В такой ситуации имеет смысл сравнивать скорость работы, объём занимаемой памяти и качество работы с моделью, где используются обычные линейные слои: таким образом, модель с уменьшенными линейными слоями будет представлять из себя trade-off качества и оптимизации, и на основе перечисленных оценок можно будет понять, стоит ли её использовать.